

Rapport

Ian Hang

March 2026

Préambule

Dans un premier temps, j'ai cherché à me former sur le **LiDAR**. Pour cela, j'ai commencé par regarder ce qui était fait sur YouTube, et j'ai demandé à ChatGPT de me proposer un plan pour apprendre à appréhender ce genre de jeux de données.

Ainsi, toutes les bibliothèques permettant de traiter de fichiers .laz m'ont été ou bien fournies par des codes existants sur Github / Reddit, ou bien par ChatGPT¹.

Par la suite, j'ai téléchargé une dalle **.laz** sur le site fournit. J'ai choisi de télécharger la dalle contenant ma maison, qui sera le **CityJSON** que j'essaierai par la suite d'obtenir.

Avant de commencer, j'ai décidé d'organiser le mini projet de la façon suivante :

- Toutes les fonctions externes dans un fichier **function.py**
- Un dossier pour les données
- Un dossier pour les sorties (GeoJSON, CityJSON)
- Un notebook pour l'analyse des données
- Des notebooks au fur et à mesure que le projet avance de manière à avoir une avancée distincte

1 Premier notebook - Analyse de données

Une fois la dalle téléchargée j'ai cherché à analyser les données, pour cela, j'utilise la bibliothèque **laspy**. Cela m'a permis d'appréhender la forme du jeu de données et d'en faire l'étude basique : Nombre de points, formats des données, altitudes, densité et classes.

2 Deuxième notebook - Délimitation des bâtiments

Après avoir appréhendé le format et le type des données. J'ai cherché à délimiter les bâtiments. Une classification par classes ayant déjà été réalisé sur le jeu de données, j'ai commencé par récupérer tous les points correspondants à des bâtiments. Il s'agit des bâtiments de classe 6 (repérés lors de l'analyse des données), qui constituent un sixième du jeu de données.

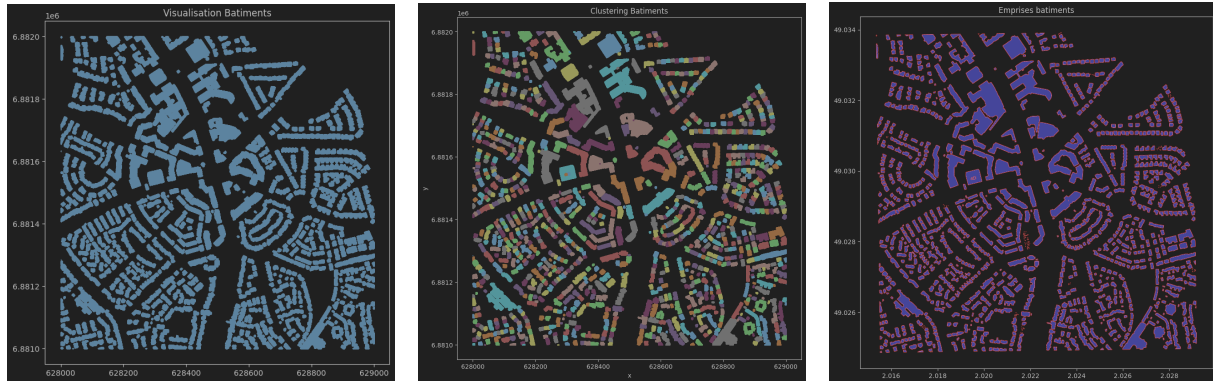
Dans un deuxième temps, j'ai cherché à délimiter les bâtiments entre eux. Pour cela, j'ai appris à utiliser la bibliothèque **sklearn**. En particulier **DBSCAN** que j'ai utilisé sur les

¹La liste des bibliothèques que j'ai découvert pour l'occasion est en annexe

coordonnées (x, y) de tous les points types bâtiments. J'ai calibré les paramètres ϵ et le nombre de points minimum à la main. Cela m'a permis d'obtenir un cluster de points par bâtiments.

Une fois que tous les bâtiments ont été délimités et stockés, j'ai voulu déterminer leur empreinte (footprint), *ie.* leur contour. Pour se faire, j'utilise la bibliothèque **alphashape** sur les coordonnées (x, y) de chaque cluster de points.

Pour sauvegarder proprement le GeoJSON, j'ai utilisé **geopanda**.



Points de classe 6

Points regroupés par clusters

Emprise des bâtiments

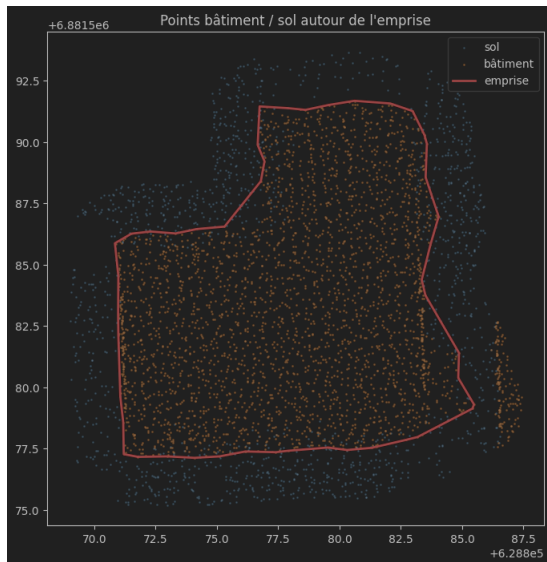
3 Troisième notebook - Modélisation de ma maison

Une fois que le fichier GeoJSON a été généré, j'ai pu visualiser le résultat sur la visionneuse **CRAIG** (que vous m'avez fait découvrir). Cela m'a permis d'obtenir l'ID de l'empreinte de ma maison. Ce sera le CityJSON que je tenterai alors de réaliser.

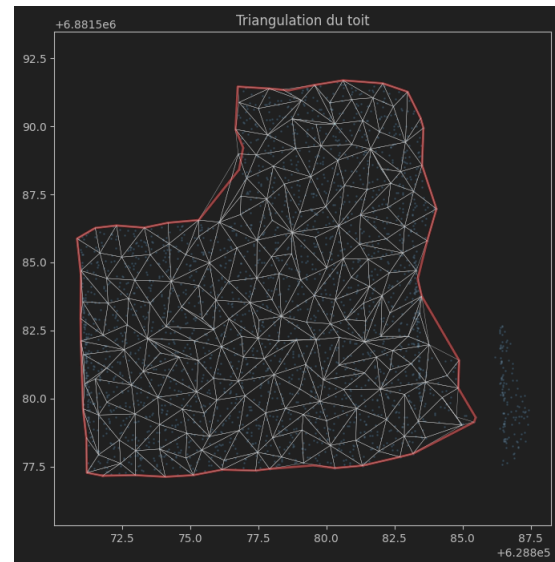
Dans un premier temps, j'ai appris à utiliser une pipeline **PDAL** pour isoler ma maison sur la tuile de $1\text{km} \times 1\text{km}$ (avec un léger buffer de 2m). Puis j'ai séparé les points sélectionnés selon leur classe pour obtenir le sol et le bâtiment.

Ensuite, ayant coordonnées des points (x, y) de l'empreinte, il m'a fallu estimer l'altitude z du début du toit. Pour cela, je me base sur l'altitude points aux alentours et je prends la médiane. Cela me permet d'avoir : d'une part le contour au sol de la maison (en prenant comme z l'altitude du sol), et d'autre part le contour au niveau du toit de la maison avec les altitudes obtenues.

Enfin pour construire le CityJSON il a fallu relier les points entre eux. Le sol est simplement un polygone formé de tous les points de l'empreinte au sol. Les murs sont des rectangles formés des points de l'empreinte au sol et des points de l'empreinte au niveau du toit. La partie difficile a été de modéliser le toit. Pour se faire, j'ai utilisé le module **scipy.spatial** qui m'a permis de faire une triangulation de Delaunay avec tous les points du toit. Ainsi j'ai pu obtenir une liste de triangles qui me serviront à paver le toit.

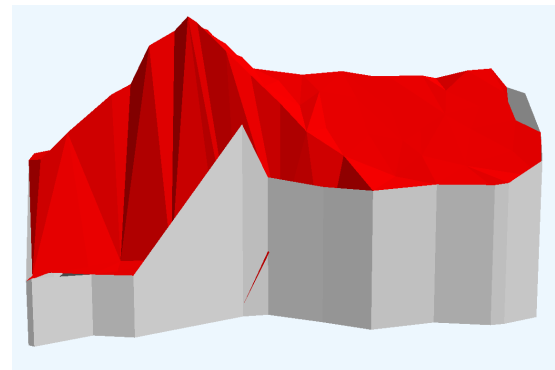
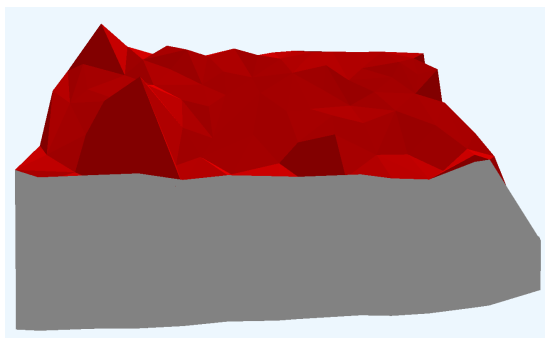


Délimitation de la maison



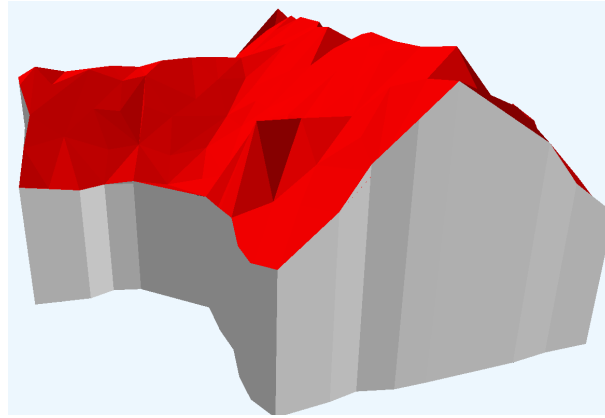
Triangulation du toit

Une fois toutes les surfaces obtenues, j'ai demandé à ChatGPT de me générer la structure json correcte pour pouvoir faire un CityJSON lisible. Le résultat est reconnaissable, mais malheureusement absolument pas sans défaut.



4 Derniers notebooks - Une version améliorée

Non satisfait du résultat, j'ai fait revoir le code à ChatGPT en lui demandant les pistes d'amélioration. Et il m'a proposé une pipeline **PDAL** plus rigoureuse et plus et plus propre du notebook précédent. Le cheminement reste le même, mais en mieux. Le résultat, bien que loin du résultat souhaité, n'a déjà plus les défauts majeurs du précédent.



Le dernier notebook consiste à réunir tous les notebooks précédents pour pouvoir réaliser tout ce que nous avons fait en une seule fois.

Conclusion

De nombreuses conclusions sont à tirer de ce travail. En premier lieu, j'aimerais revenir sur le fait que j'ai pratiquement appris intégralement comment traiter des données de type LiDAR. Ce qui d'une part me satisfait, mais d'autre part m'oblige à rappeler qu'une IA générative est passé après *pratiquement* l'intégralité du code, pour l'orienter, le corriger, ou par moment le créer.

Pour revenir sur le GeoJSON, j'en suis satisfait. Même si l'on peut observer par endroit du bruit et des aberrations, la forme de la ville est la bonne, et une calibration plus précise de la clusterisation pourrait permettre de gommer ses défauts.

Quant au CityJSON, il me faut reconnaître être encore loin de l'exemple fournit. Je ne suis pas parvenu dans les temps impartis à lisser la figure pour obtenir un bâtiment aussi propre. En conséquence, je suis pas non plus en mesure de déterminer certains attributs, comme la pente ou la surface du toit. (Bien que je pourrais en sommant l'aire de chaque triangle, nous aurions un résultat amplement surestimé.). La forme de la maison était peut-être également un mauvais choix de ma part. En effet, étant en forme de "L", il est impossible de reconstruire le toit en seulement deux pans, ce que j'ai tenté sans succès.

A Liste des bibliothèques découvertes grâce à ChatGPT, Github et Reddit

- laspy
- alphashape
- geopandas
- sklearn.cluster
- pdal
- scipy.spatial
- shapely.geometry

Ainsi, à chaque fois que ces bibliothèques sont mentionnées, il s'agit de bibliothèques que j'ai appris à utiliser pour mener à bien ce projet.

B readme

Si vous souhaitez tester le code de votre côté vous pouvez :

- Ou bien tester les notebooks un à un en inscrivant simplement le bon **laz_path** au début du notebook.
- Ou bien tester l'intégralité du code avec le 4ème notebook : **04_pipeline_complet**²

Tous les résultats seront stockés dans le dossier **output**.

²Attention à l'id du bâtiment, ma maison correspond au 933, qui peut ne pas exister en fonction du .laz fournit.